

15-Day AI / Gen AI / Agentic AI Learning Plan

Audience: Senior professional with 12 years of Azure Cloud and Data experience moving into AI engineering roles. **Goal:** Working fluency in classical ML concepts, Gen AI, RAG, Agentic AI, and the current Microsoft AI stack (Microsoft Foundry, Agent Framework 1.0, Azure OpenAI, Azure AI Search), with interview-ready depth.

How to Use This Plan

- **Time budget:** 4-6 hours/day. The plan compresses where your existing Azure/data skills overlap, and expands where the GenAI/agentic concepts are new.
 - **Structure per day:** Concepts → Explanation → Hands-on → Interview Q&A → Official references.
 - **Mindset:** You don't need to write training code from scratch for ML algorithms — you need to *reason* about them. For GenAI and agents, hands-on is non-negotiable.
 - **Tooling to install on Day 0:** Python 3.11+, VS Code with Python + Jupyter extensions, Azure CLI, an Azure subscription (for Foundry/OpenAI access — apply early; Azure OpenAI access can take a couple of days), Docker Desktop, Git, Postman or similar.
-

Plan at a Glance

Phase	Days	Theme
1	1-3	Classical ML & Deep Learning foundations (refresher + transformers)
2	4-6	Gen AI core — LLMs, embeddings, prompting, fine-tuning
3	7-8	RAG & Vector Search end-to-end
4	9-11	Agentic AI — patterns, tools, multi-agent orchestration
5	12-13	Microsoft Foundry & Microsoft Agent Framework
6	14	LLMOps, evaluation, responsible AI
7	15	Capstone build + mock interviews

PHASE 1 — Classical ML & Deep Learning Foundations

Day 1 — Supervised & Unsupervised Learning, Model Evaluation

Concepts to master

- **Supervised:** regression (linear, polynomial, regularized — Ridge, Lasso, ElasticNet), classification (logistic regression, decision trees, random forests, gradient boosting — XGBoost, LightGBM, CatBoost).
- **Unsupervised:** clustering (K-Means, DBSCAN, hierarchical, HDBSCAN), dimensionality reduction (PCA, t-SNE, UMAP).
- **Evaluation metrics:** classification — accuracy, precision, recall, F1, ROC-AUC, PR-AUC, log loss. Regression — MAE, RMSE, MAPE, R^2 . Confusion matrix interpretation.
- **Bias-variance tradeoff**, overfitting/underfitting, regularization mechanics, cross-validation (k-fold, stratified, time-series split, group k-fold).

Explanation (key ideas)

Regression predicts a continuous value; **classification** predicts a label. The choice of *loss function* (MSE, MAE, log loss, hinge) and *metric* (RMSE vs MAE; precision vs recall) should be driven by business cost asymmetry, not convention.

Precision asks: of what I flagged positive, how many really are? Use it when false positives are expensive (spam, fraud alerts triggering human review). **Recall** asks: of all true positives, how many did I catch? Use it when false negatives are dangerous (cancer screening). The **F1 score** balances them, but in practice plot the **PR curve** and pick the threshold that matches your cost ratio.

Regularization (L1 = Lasso, L2 = Ridge, ElasticNet = both) penalizes large coefficients to control variance. L1 also performs feature selection by zeroing weights — useful when you suspect many features are irrelevant.

Hands-on

- Take any tabular dataset (Kaggle Titanic, California Housing, or your own). Build three models (logistic regression, random forest, XGBoost), do proper CV, and produce a comparison table with the right metrics.
- Plot ROC and PR curves; choose a threshold based on a stated cost ratio.

Interview Q&A

Q1. When would you choose MAE over RMSE, and vice versa? A. RMSE squares errors, so it punishes large mistakes disproportionately — use it when big misses carry outsized business cost (e.g., demand forecasting where a 100-unit miss is far worse than ten 10-unit misses). MAE

treats errors linearly — better when error cost is roughly proportional, or when you need robustness to outliers. MAPE is interpretable as a percentage but breaks near zero and is asymmetric. For asymmetric costs, quantile loss directly.

Q2. Explain bias-variance tradeoff with a concrete example. A. A linear regression on non-linear data has high bias (it can't fit the curve) and low variance (small changes in training data won't change predictions much). A deep, unpruned decision tree has low bias (it can fit anything) but high variance (it memorizes noise). Regularization (L1/L2 in linear models; `max_depth`, `min_samples_leaf` in trees) trades a little bias for a large reduction in variance — the sweet spot is found via cross-validated hyperparameter search.

Q3. Your model has 99% accuracy but business says it's useless. What happened? A. Likely a severe class imbalance. If 99% of cases are negative, predicting all-negative gives 99% accuracy with zero recall. The fix: switch to precision/recall/F1 or PR-AUC; resample (SMOTE, class weights); choose a threshold based on the PR curve and business cost ratio; consider cost-sensitive learning.

Q4. How do you choose between K-Means and DBSCAN? A. K-Means assumes spherical clusters of similar size and needs `k` upfront — fast but rigid. DBSCAN handles arbitrary cluster shapes, doesn't need `k`, and identifies noise points (good for anomaly detection), but tuning `eps` is hard and it struggles with varying densities. For varying densities use HDBSCAN. In high dimensions, reduce with UMAP first because distance metrics lose meaning.

Official References

- Scikit-learn user guide — supervised learning: https://scikit-learn.org/stable/supervised_learning.html
 - Scikit-learn user guide — clustering: <https://scikit-learn.org/stable/modules/clustering.html>
 - Scikit-learn model evaluation metrics: https://scikit-learn.org/stable/modules/model_evaluation.html
 - Google ML Crash Course — Logistic Regression & Classification: <https://developers.google.com/machine-learning/crash-course/classification>
 - Microsoft Learn — Create ML models module: <https://learn.microsoft.com/en-us/training/paths/create-machine-learn-models/>
-

Day 2 — Feature Engineering, Imbalance, Pipelines

Concepts

- Encoding categorical variables (one-hot, ordinal, target/mean encoding with smoothing, hashing trick, embeddings).
- Numerical transformations (scaling, log/Box-Cox, binning, interactions).

- Handling missing values (imputation strategies, "missingness as feature").
- Class imbalance — class weights, SMOTE, undersampling, focal loss.
- Feature selection (filter, wrapper, embedded methods; SHAP-based importance).
- Pipelines and leakage prevention (fit on train only, encode in pipeline, time-aware splits).

Explanation

High-cardinality categoricals (ZIP code, SKU with 50k+ levels) defeat one-hot encoding. Use **target encoding** with smoothing and **out-of-fold** computation to prevent leakage (encoding using future labels), or **hashing trick** when collisions are acceptable. CatBoost handles high cardinality natively via ordered target statistics — one reason it's my default for messy tabular data.

Target leakage is the silent killer: a feature that uses information unavailable at prediction time. Examples: aggregations computed over the full dataset (including test), or "days since last purchase" computed using a column updated *after* the prediction event. The fix is rigorous separation in your pipeline.

Hands-on

- Build an `sklearn` Pipeline that does imputation → encoding → scaling → model in one object. Verify the test set is transformed using parameters fit only on the train set.
- Take an imbalanced dataset and compare baseline, class-weighted model, and SMOTE.

Interview Q&A

Q1. Walk me through how you encode a categorical feature with 50,000 distinct values. A. One-hot is out — it would create 50k sparse columns. Options ranked by typical preference: (1) target encoding with k-fold smoothing — replace category with the smoothed mean of the target for that category, computed out-of-fold to prevent leakage; (2) frequency encoding — replace with count; preserves information cheaply; (3) hashing trick — fixed dimension with controlled collisions, good when features are too dynamic to store a vocab; (4) embeddings — learned dense representations if I'm feeding a neural net; (5) group rare levels into "other".

Q2. How do you prevent data leakage in a pipeline? A. Three rules: (1) never fit any transformer on test or full data — only on train; (2) compute statistics (target encoding, scaling means, aggregations) using only past or in-fold data; (3) for time series, never sample randomly — always split chronologically. I encode this in sklearn Pipelines / ColumnTransformers so transformation order is enforced.

Q3. SMOTE vs class weights — which do you reach for first? A. Class weights, because they require no synthetic data and don't change the input distribution — most models support them natively (`class_weight='balanced'`). SMOTE is useful when models can't accept weights or when the minority class is so small that the boundary is poorly learned. SMOTE on high-

dimensional or categorical data is risky — synthetic points can be unrealistic. I always validate with the same imbalance ratio in CV that exists in production.

Official References

- scikit-learn — preprocessing & feature engineering: <https://scikit-learn.org/stable/modules/preprocessing.html>
 - scikit-learn — pipelines: <https://scikit-learn.org/stable/modules/compose.html>
 - imbalanced-learn (SMOTE & friends): <https://imbalanced-learn.org/stable/>
 - Microsoft Learn — Train and evaluate models: <https://learn.microsoft.com/en-us/training/paths/train-evaluate-models/>
-

Day 3 — Deep Learning & Transformers Crash Course

Concepts

- Neural network basics — perceptron, activation functions (ReLU, GELU, sigmoid, softmax), backprop, optimizers (SGD, Adam, AdamW), loss functions.
- Regularization in DL — dropout, batch/layer norm, weight decay, early stopping.
- Architectures at a glance — CNN (vision), RNN/LSTM (sequence, mostly historical now), **Transformers** (the foundation of all modern LLMs).
- Transformers in depth — tokenization, embeddings, positional encoding, **self-attention**, multi-head attention, encoder vs decoder, masked language modeling vs causal LM.

Explanation

The transformer's key innovation is **self-attention**: every token attends to every other token in the input, weighted by learned relevance. This replaces recurrence (RNNs) and gives the model parallel processing plus the ability to handle long-range dependencies.

- **Encoder-only** (BERT) — bidirectional context; great for understanding (classification, embeddings).
- **Decoder-only** (GPT family) — causal (left-to-right); great for generation. All modern chat LLMs are decoder-only.
- **Encoder-decoder** (T5, BART) — used for translation/summarization originally; still relevant for some tasks.

Attention complexity is $O(n^2)$ in sequence length — this is why context windows are expensive and why innovations like Flash Attention, sliding window attention, and ring attention exist.

Hands-on

- Read "The Illustrated Transformer" (Jay Alammar) — non-negotiable.
- In Hugging Face, load a small BERT model and generate embeddings; load a small GPT-2 model and generate text. Inspect tokens, attention patterns, hidden states.

Interview Q&A

Q1. Explain self-attention in one paragraph. A. For each token, attention computes three projections — Query, Key, Value. The token's Query is dot-producted with every other token's Key to produce a similarity score; scores are softmax-normalized to become weights; the output for that token is the weighted sum of all Values. The result: each token's new representation is a mixture of every other token, weighted by relevance. Multi-head attention runs this in parallel with different projection matrices to capture different relationship types.

Q2. Why are decoder-only models the dominant LLM architecture today? A. Causal (left-to-right) attention lets a single architecture do both pretraining (predict next token on huge text corpora) and generation (the same next-token prediction at inference time). It's simple, scales beautifully, and the same model can be prompted to do classification, generation, reasoning, code — emergent capabilities arrive with scale. Encoder-decoder shines for specific seq2seq tasks but decoder-only's flexibility plus scaling wins for general-purpose AI.

Q3. What's the attention bottleneck and what techniques address it? A. Standard attention is $O(n^2)$ in sequence length for compute and memory — quadratic blowup limits context length and inference speed. Techniques: **Flash Attention** (IO-aware exact attention that fits attention in SRAM); **sliding window attention** (each token attends only to a local window — Mistral); **sparse attention** patterns (Longformer, BigBird); **linear attention approximations** (Performer); **MoE** (mixture of experts) to scale params without proportional compute; **KV caching** to avoid recomputation during autoregressive generation; **paged attention** (vLLM) for memory efficiency at serve time.

Q4. Tokenization — why does it matter? A. LLMs operate on tokens, not characters or words. Modern tokenizers (BPE in GPT, SentencePiece in many open models) split text into subword units. Tokenization affects: (1) cost (you're billed per token); (2) context window utilization; (3) model performance on rare words, code, and non-English languages; (4) numerical reasoning (numbers get split inconsistently, which is why early LLMs were bad at arithmetic). Always know your model's tokenizer and watch token counts in production.

Official References

- The Illustrated Transformer (Jay Alammar): <https://jalammar.github.io/illustrated-transformer/>
- "Attention is All You Need" paper: <https://arxiv.org/abs/1706.03762>
- Hugging Face — Transformers course: <https://huggingface.co/learn/llm-course/chapter1/1>
- Hugging Face Transformers docs: <https://huggingface.co/docs/transformers/index>

- Stanford CS25 — Transformers United (lectures, optional deep dive):
<https://web.stanford.edu/class/cs25/>
-

PHASE 2 — Gen AI Core

Day 4 — LLMs, Tokens, Embeddings, Context Windows

Concepts

- LLM lifecycle — pretraining, supervised fine-tuning (SFT), preference tuning (RLHF, DPO).
- Inference parameters — temperature, top-p, top-k, frequency/presence penalty, max tokens, stop sequences, seed.
- Embeddings — semantic vectors, similarity (cosine, dot product, Euclidean), embedding models (OpenAI text-embedding-3, Cohere, BGE, E5, Nomic), Matryoshka embeddings.
- Context window management — token budgeting, lost-in-the-middle, sliding window, summarization-based memory.
- Frontier model families and trade-offs (OpenAI GPT-4o/o-series, Anthropic Claude, Google Gemini, Meta Llama, Mistral, Microsoft Phi).

Explanation

Temperature controls the softmax sharpness — 0 is greedy (always pick the most likely token), 1+ is more random. **Top-p (nucleus)** samples from the smallest set of tokens whose cumulative probability exceeds p. For deterministic, factual tasks set temperature = 0 and use a seed; for creative tasks raise temperature.

Embeddings are dense vectors capturing semantic meaning — `text-embedding-3-large` is 3072-dimensional. Similar meanings → similar vectors. Cosine similarity is the standard metric. **Matryoshka embeddings** are trained so you can truncate (e.g., from 3072 to 512 dimensions) with graceful quality degradation — a huge cost saver.

Lost-in-the-middle: LLMs attend more strongly to information at the start and end of the prompt. Put the most relevant content first (or repeat it at the end).

Hands-on

- Call Azure OpenAI Chat Completions API from Python. Experiment with temperature 0, 0.7, 1.2 on the same prompt — observe the distribution of outputs.
- Generate embeddings for 20 sentences (5 topics × 4 paraphrases each); compute pairwise cosine similarity; verify same-topic sentences cluster together.

Interview Q&A

Q1. Temperature vs top-p — when do you use which? A. Both shape the output distribution. Temperature rescales logits before softmax; top-p truncates the sampling pool by cumulative probability. Setting one is usually enough. For deterministic outputs (extraction, classification, code) use temperature = 0. For creative writing, temperature = 0.7-1.0 with top-p = 0.9. I rarely use both aggressively at once — you lose control. Set a seed when you need reproducibility for evals.

Q2. How do you pick an embedding model? A. (1) Run it on *your* retrieval task — MTEB benchmarks are a starting point, not the answer. Measure recall@k and NDCG on a labeled query set. (2) Consider dimensions vs cost — higher dims carry more signal but cost more to store and search; Matryoshka models let you truncate. (3) Context length — if your chunks are long, you need an 8k+ context embedder. (4) Domain match — generic English embeddings underperform on code, legal, or non-English; consider domain-tuned or multilingual models. (5) Cost vs latency — API (OpenAI, Cohere) vs self-hosted (BGE, E5, Nomic). For high-volume offline indexing, self-hosted often wins.

Q3. What is "lost in the middle" and how do you mitigate it? A. LLMs systematically attend more to information at the beginning and end of the context, less to the middle. In a long RAG context with 20 chunks, the most relevant chunk in position 10 may be ignored. Mitigations: (1) rerank so the best chunk is first or last; (2) trim aggressively — better 5 chunks than 20; (3) summarize middle content; (4) use models with stronger long-context performance (recent frontier models have improved here).

Q4. Explain the difference between SFT, RLHF, and DPO. A. After pretraining, **SFT** (supervised fine-tuning) trains the model on high-quality input/output pairs to teach format and instruction-following. **RLHF** (reinforcement learning from human feedback) trains a reward model from human preference comparisons, then uses PPO to align the LLM to maximize that reward — this is what made ChatGPT feel "good." **DPO** (Direct Preference Optimization) achieves the same alignment goal with simpler math — it directly optimizes the policy on preference pairs, no separate reward model or RL loop. DPO is now widely preferred for open-model alignment because it's more stable and easier to implement.

Official References

- OpenAI API reference: <https://platform.openai.com/docs/api-reference>
- OpenAI text generation guide: <https://platform.openai.com/docs/guides/text-generation>
- Azure OpenAI Service documentation: <https://learn.microsoft.com/en-us/azure/ai-services/openai/>
- Azure OpenAI — embeddings concept: <https://learn.microsoft.com/en-us/azure/ai-services/openai/concepts/understand-embeddings>
- Anthropic Claude API docs: <https://docs.claude.com/en/api>

- MTEB (Massive Text Embedding Benchmark): <https://huggingface.co/spaces/mteb/leaderboard>
 - "Lost in the Middle" paper: <https://arxiv.org/abs/2307.03172>
-

Day 5 — Prompt Engineering

Concepts

- Prompt anatomy — system message, role, task, constraints, format, examples.
- Techniques — zero-shot, few-shot, **Chain-of-Thought (CoT)**, **self-consistency**, **ReAct**, **Tree-of-Thoughts**, role prompting, decomposition.
- Structured outputs — JSON mode, function calling, JSON Schema, Pydantic + instructor.
- Prompt patterns — extraction, classification, summarization, transformation, agentic tool use.
- Prompt injection & defense — system prompt isolation, input sanitization, output validation.
- Prompt versioning, evaluation, and iteration.

Explanation

A good prompt has structure: **role + task + constraints + output format**. Few-shot examples are powerful when format matters or the task is novel; pick examples that span your input distribution.

Chain-of-Thought ("think step by step" or structured scratchpads) dramatically improves reasoning on math, logic, and multi-step tasks. **Self-consistency** runs CoT multiple times and majority-votes — boosts accuracy at the cost of more tokens. **ReAct** interleaves reasoning and action (tool calls) — the foundation of modern agents.

Structured outputs are the difference between a demo and production. Use JSON mode or function calling with strict schemas; on parse failure, retry with the error embedded in the prompt.

Prompt injection is the GenAI equivalent of SQL injection — user input or retrieved content includes instructions like "ignore previous instructions and..." Defenses: keep system prompts in a separate role; treat retrieved content as data, never instructions; validate outputs against a schema; for high-stakes actions, require an additional verification step.

Hands-on

- Take a real task (e.g., extract structured data from invoices, classify support tickets). Write three prompt versions: zero-shot, few-shot, CoT. Run each over 20 examples. Score

accuracy.

- Implement structured output using OpenAI function calling or `instructor` + Pydantic.
- Try a prompt injection attack on your own system and fix the defenses.

Interview Q&A

Q1. Walk me through how you'd systematically improve a prompt. A. Treat prompts as code: version-controlled, tested, evaluated. Build an eval set of input/expected-output pairs (50–200 examples covering the distribution). Iterate prompt variants, run each over the set, track metrics (accuracy, latency, cost). Identify failure patterns — clustering errors often reveals a missing instruction or a confused edge case. Avoid changing two things at once. Maintain a prompt changelog. Log production inputs and sample failures back into the eval set.

Q2. When does Chain-of-Thought help, and when does it hurt? A. CoT helps for multi-step reasoning — math, logic, planning, code debugging. It hurts when (1) the task is simple lookup or extraction (CoT adds latency and tokens for no gain); (2) the model is small (CoT capability is emergent at scale; small models can produce confident but wrong reasoning); (3) the output format must be strict and free of explanation. Modern frontier models often do internal reasoning ("thinking" modes), making explicit CoT prompting less necessary on hard tasks.

Q3. How do you guarantee a model outputs valid JSON? A. Layered approach: (1) use the model's structured-output mode (OpenAI's `response_format` with JSON Schema, or strict mode); (2) provide a few-shot example of the exact format; (3) validate the output against a Pydantic/JSON Schema; (4) on validation failure, retry with the error message included in a follow-up prompt — usually fixes it on the second try; (5) for absolute reliability, use constrained decoding (grammars like Outlines or libraries that mask invalid tokens during generation).

Q4. How do you defend against prompt injection? A. Multiple layers: (1) **separate roles** — system instructions in `system`, user input in `user`, retrieved content clearly delimited and labeled as data not instructions; (2) **content safety filters** on inputs and outputs (Azure AI Content Safety has indirect-attack detection); (3) **principle of least privilege** — the LLM doesn't have direct access to dangerous tools; sensitive actions require deterministic checks or human approval; (4) **output validation** — schema, allowlists, regex for sensitive patterns; (5) **monitor and red-team** with a known set of injection payloads in CI; (6) for retrieval, sanitize untrusted documents (e.g., remove anything that looks like instructions) when indexing.

Official References

- OpenAI prompt engineering guide: <https://platform.openai.com/docs/guides/prompt-engineering>
- Anthropic prompt engineering guide: <https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/overview>
- Azure OpenAI prompt engineering techniques: <https://learn.microsoft.com/en-us/azure/ai-services/openai/concepts/prompt-engineering>

- OpenAI structured outputs: <https://platform.openai.com/docs/guides/structured-outputs>
 - Anthropic structured outputs / tool use: <https://docs.claude.com/en/docs/agents-and-tools/tool-use/overview>
 - Chain-of-Thought paper: <https://arxiv.org/abs/2201.11903>
 - ReAct paper: <https://arxiv.org/abs/2210.03629>
 - Prompt Injection — OWASP LLM Top 10: <https://genai.owasp.org/llm-top-10/>
-

Day 6 — Fine-tuning, PEFT, RLHF/DPO

Concepts

- When to fine-tune vs prompt vs RAG.
- Full fine-tuning vs **PEFT** (parameter-efficient fine-tuning) — **LoRA**, **QLoRA**, adapters, prefix tuning.
- Instruction tuning (SFT), preference tuning (RLHF with PPO, DPO, KTO).
- Data preparation — quality > quantity, schema (chat format), train/val split, deduplication.
- Evaluation of fine-tuned models — benchmark vs held-out task set, regression on general capability.
- Azure OpenAI fine-tuning and serverless fine-tuning in Foundry.

Explanation

Decision tree:

- **Need facts?** → RAG.
- **Need behavior / style / format?** → Fine-tune.
- **Need both?** → Both — they're complementary, not competing.

LoRA freezes the base model and adds small low-rank update matrices to attention/feedforward layers. You train ~0.1-1% of the parameters; the result is a tiny adapter (often <100 MB) that's swapped at inference. **QLoRA** combines LoRA with 4-bit quantization of the base model — lets you fine-tune 70B models on a single A100. Memory savings are massive; performance is competitive with full fine-tuning for most tasks.

Data quality matters more than quantity. 500 high-quality examples often beat 50,000 noisy ones. Deduplicate, fix label errors, ensure diversity, and reserve 10-20% for evaluation.

Hands-on

- Fine-tune a small open model (e.g., Phi-3 or Llama-3-8B) with LoRA on a domain dataset using Hugging Face PEFT library, or do a fine-tune job in Azure AI Foundry.

- Compare before/after on a held-out eval set; also check it didn't regress on general capability.

Interview Q&A

Q1. RAG or fine-tune — how do you decide? A. RAG wins when: (a) knowledge changes frequently; (b) you need citations and traceability; (c) data is too small or noisy to fine-tune; (d) you need access controls per document. Fine-tuning wins when: (a) you need a *style, format, or reasoning pattern*; (b) prompt length is bloating costs; (c) you have a stable, well-labeled task with thousands of examples; (d) you need lower latency than a long RAG prompt allows. In practice many production systems use both: fine-tune the base behavior, RAG for facts.

Q2. Explain LoRA and why it's so popular. A. LoRA hypothesizes that the *change* needed to adapt a pretrained model to a new task is low-rank. It freezes the original weight matrix W and adds a learned update $\Delta W = BA$, where B and A are small ($\text{rank } r \ll \text{original dim}$). You train only B and A — often $<1\%$ of parameters. Benefits: (1) low memory; (2) small adapters (<100 MB) that can be swapped per task; (3) no catastrophic forgetting on the base; (4) multiple adapters can be hot-swapped at serve time. QLoRA adds 4-bit quantization to fit even bigger base models.

Q3. What goes wrong with fine-tuning if you're not careful? A. (1) **Catastrophic forgetting** — model loses general capability; mitigate by mixing some general data in training and evaluating on broad benchmarks, not just your task. (2) **Overfitting** — too many epochs on too little data; watch validation loss, stop early. (3) **Data quality issues** — label noise, leakage from eval set into train, format inconsistency. (4) **Hidden distribution shift** — your fine-tune set doesn't match production inputs. (5) **Cost surprises** — fine-tuned model inference may be priced higher than base; verify before committing. (6) **Skill regression** — gains on your task but loss elsewhere; always run a regression eval.

Q4. DPO vs RLHF — what's the practical difference? A. Both align a model to human preferences. RLHF trains a separate reward model from preference pairs, then uses PPO to optimize the LLM against that reward — multi-stage, hyperparameter-sensitive, unstable. DPO derives a closed-form objective that directly optimizes the policy on preference pairs without an explicit reward model or RL — simpler, more stable, fewer hyperparameters. For most open-model alignment, DPO is now the default. RLHF-style is still used by frontier labs with sophisticated infra. KTO is a related variant that doesn't even require paired preferences — just thumbs-up/down.

Official References

- LoRA paper: <https://arxiv.org/abs/2106.09685>
- QLoRA paper: <https://arxiv.org/abs/2305.14314>
- Hugging Face PEFT library: <https://huggingface.co/docs/peft/index>
- DPO paper: <https://arxiv.org/abs/2305.18290>

- Azure OpenAI fine-tuning docs: <https://learn.microsoft.com/en-us/azure/ai-services/openai/how-to/fine-tuning>
 - OpenAI fine-tuning guide: <https://platform.openai.com/docs/guides/fine-tuning>
 - Microsoft Foundry — fine-tuning concept: <https://learn.microsoft.com/en-us/azure/ai-foundry/concepts/fine-tuning-overview>
-

PHASE 3 — Retrieval-Augmented Generation & Vector Search

Day 7 — RAG Architecture End-to-End

Concepts

- RAG pipeline — ingest → parse → chunk → embed → store → retrieve → (rerank) → generate.
- Chunking strategies — fixed size, recursive, semantic, structure-aware (Markdown/HTML/PDF).
- Document parsing — PDFs (PyMuPDF, Unstructured), Office docs, tables (Azure Document Intelligence), images (multimodal embeddings).
- Retrieval — dense (vector), sparse (BM25), **hybrid**, with metadata filters.
- Reranking — cross-encoders, Cohere Rerank, Azure AI Search semantic ranker.
- Grounding, citations, attribution.

Explanation

Naïve RAG (embed-and-retrieve, top-k, stuff into context) works in demos and breaks in production. Advanced techniques per stage:

Chunking: Don't blindly split every 1000 characters. Use **recursive splitters** that respect paragraphs, sentences, and structural boundaries. For long documents, **semantic chunking** (split where sentence embeddings shift) preserves meaning. Aim for 256–512 token chunks with 10–20% overlap. Index parent-document references so you can retrieve small chunks but inject larger context.

Hybrid retrieval: Dense embeddings capture semantic similarity; BM25 captures exact lexical matches (names, IDs, rare terms). Most production systems combine them (Reciprocal Rank Fusion or weighted score). Azure AI Search supports this natively plus a semantic ranker.

Reranking: Top-k by embedding similarity is fast but coarse. A cross-encoder reranker scores query-document pairs jointly and is much more accurate. Pattern: retrieve top-50 cheaply, rerank to top-5 with a cross-encoder.

Hands-on

- Build a RAG over your own documents (e.g., team wiki, PDFs). Use LangChain or LlamaIndex initially for speed, then strip back to direct API calls.
- Compare: top-5 dense only vs top-5 hybrid vs top-50 hybrid + rerank-to-5. Measure on 20 hand-labeled queries.

Interview Q&A

Q1. Walk through a production RAG architecture. A. Ingestion layer: scheduled pipeline (ADF or Fabric) pulls documents, normalizes (Document Intelligence for PDFs/tables/images), chunks with structure awareness, embeds via Azure OpenAI embeddings, writes to **Azure AI Search** with both vector index and BM25 fields plus metadata for filtering. Query layer: receive query, optional query rewrite/HyDE, hybrid retrieve top-50, rerank with semantic ranker to top-5, build prompt with chunks + citations + instructions to ground or refuse, call the LLM, return response with source citations. Cross-cutting: managed identities, private endpoints, content safety on input and output, App Insights tracing, eval harness in CI, security-trimming for per-user document access.

Q2. What are the most common RAG failure modes? A. (1) **Bad chunking** — fixed sizes split semantic units; fix with recursive or semantic chunking. (2) **Retrieval miss** — query and document use different terms (e.g., "401k" vs "retirement plan"); fix with hybrid search, query expansion, HyDE. (3) **Top-k by similarity $\neq$ relevance** — add a reranker. (4) **Lost in the middle** — too many chunks; trim or reorder. (5) **No grounding enforcement** — model hallucinates; strict prompt that says "answer only from context, otherwise say 'I don't know,'" with required citations. (6) **No eval** — quality degrades silently; build RAGAS-style metrics in CI. (7) **Stale index** — documents change; automate reindexing on source events.

Q3. How do you handle document-level access control in RAG? A. **Security trimming at retrieval time**, not generation time. Index each chunk with the ACL of its source (allowed user IDs, groups). At query time, filter the search to chunks the requesting user is authorized to see. Azure AI Search supports security filters via a field. Never rely on the LLM to "not mention" content — if the context contains it, assume it can be exfiltrated. Pair with audit logging of who queried what.

Q4. What's HyDE and when does it help? A. **Hypothetical Document Embeddings**. Instead of embedding the user's question, ask the LLM to generate a hypothetical answer, then embed *that* and search. The hypothetical answer is in the same "shape" as documents, so it matches better in embedding space. Helps when queries are short or under-specified and documents are long-form. Costs an extra LLM call. Less needed today as embedding models have improved at query-doc asymmetry; still a useful tool when retrieval underperforms.

Official References

- Azure AI Search overview: <https://learn.microsoft.com/en-us/azure/search/search-what-is-azure-search>
 - Azure AI Search — RAG concepts: <https://learn.microsoft.com/en-us/azure/search/retrieval-augmented-generation-overview>
 - Azure AI Search — hybrid retrieval: <https://learn.microsoft.com/en-us/azure/search/hybrid-search-overview>
 - Azure AI Search — semantic ranker: <https://learn.microsoft.com/en-us/azure/search/semantic-search-overview>
 - Azure Document Intelligence: <https://learn.microsoft.com/en-us/azure/ai-services/document-intelligence/>
 - LangChain RAG tutorial: <https://python.langchain.com/docs/tutorials/rag/>
 - LlamaIndex documentation: <https://docs.llamaindex.ai/>
 - HyDE paper: <https://arxiv.org/abs/2212.10496>
 - RAG (original) paper: <https://arxiv.org/abs/2005.11401>
-

Day 8 — Vector Databases & Advanced RAG

Concepts

- Vector index algorithms — exact (brute-force), **HNSW**, **IVF**, **IVF-PQ**, ScaNN.
- Trade-offs — recall vs latency vs memory vs build time.
- Vector DBs — **Azure AI Search**, **Cosmos DB for NoSQL/Mongo (vector index)**, **PostgreSQL pgvector**, Pinecone, Weaviate, Qdrant, Milvus, Chroma.
- Advanced RAG patterns — query rewriting, **multi-query** retrieval, **HyDE**, **parent-document retrieval**, **sentence-window retrieval**, **GraphRAG**, **agentic RAG**.
- RAG evaluation — **RAGAS** (faithfulness, answer relevance, context precision, context recall), groundedness, hallucination detection.

Explanation

HNSW (Hierarchical Navigable Small World) — graph-based, layered. Excellent recall + low latency; high memory (graph structure is big). Default for most production. Knobs: `M` (graph connectivity) and `efConstruction` / `efSearch` (build/query exploration).

IVF (Inverted File Index) — partition vectors into clusters; query probes the nearest clusters. Lower memory; recall depends on `nprobe`. Used at very large scale.

IVF-PQ — IVF plus Product Quantization (compressed vectors). Massive memory savings; some recall loss. Foundation of FAISS at billion-vector scale.

GraphRAG (Microsoft): builds a knowledge graph from documents during indexing (entities, relationships, communities). At query time it combines graph traversal with vector retrieval. Excels at multi-hop questions and "what's the bigger picture?" questions where naïve RAG struggles.

Agentic RAG: an agent decides *whether* to retrieve, *what* to retrieve, *when to stop*, and may iterate (retrieve → analyze → refine query → retrieve again). Higher quality on complex queries; more expensive.

Hands-on

- Stand up **Azure AI Search** with a vector index, integrated vectorization, and semantic ranker. Index a corpus and query it three ways: vector only, hybrid, hybrid + semantic ranker. Compare results.
- Implement **RAGAS** evaluation over 30 labeled queries. Get baseline metrics; iterate one retrieval improvement; remeasure.

Interview Q&A

Q1. Compare HNSW, IVF, and IVF-PQ. A. **HNSW** — graph-based; highest recall + lowest latency at small/medium scale; memory-hungry (graph is large); slow to build; great default. **IVF** — partition + probe; lower memory; recall scales with `nprobe`, which trades latency. **IVF-PQ** — IVF plus quantization of vectors; massive memory savings; recall loss is the cost; the choice at billion+ scale. ScaNN — Google's variant with anisotropic quantization; very strong benchmarks. In Azure AI Search, the underlying ANN is HNSW. I tune `M` and `efSearch` against a held-out query set, optimizing recall@k subject to a latency budget.

Q2. When is GraphRAG worth the complexity? A. When questions are multi-hop or require understanding relationships across documents — "How did our product strategy in 2023 lead to the 2025 reorganization?" — naïve RAG retrieves disconnected chunks and the LLM struggles to synthesize. GraphRAG indexes entities and relationships explicitly, supports traversal, and surfaces global summaries. Costs: heavier indexing pipeline, more storage, harder to update incrementally. I default to vanilla RAG and reach for GraphRAG when evaluation shows the failure pattern is multi-hop reasoning.

Q3. How do you evaluate a RAG system? A. Layered: (1) **Retrieval metrics** — recall@k, MRR, NDCG on a labeled query set. (2) **RAGAS** — **faithfulness** (response grounded in context), **answer relevance** (response addresses the question), **context precision** (retrieved chunks are relevant), **context recall** (relevant chunks were retrieved given a reference answer). (3) **Groundedness / hallucination** — LLM-as-judge checks each claim against context. (4) **End-to-**

end task success on a held-out evaluation set. (5) **Production telemetry** — user thumbs, follow-up rate, time-to-resolution. Wire into CI so prompt or retriever changes can't silently regress.

Q4. What's "parent-document retrieval"? A. Index small chunks for precision (~256 tokens) but at retrieval time return the *parent* (e.g., the surrounding section or full document). Best of both worlds — tight match on relevance, broader context for generation. Stored as a parent-child mapping in the index. Variant: **sentence-window retrieval** retrieves a single sentence but expands $\pm N$ sentences around it for context. Both reduce the "fragmented context" problem of naïve chunking.

Official References

- Azure AI Search — vector search overview: <https://learn.microsoft.com/en-us/azure/search/vector-search-overview>
 - Azure AI Search — choose a vector algorithm: <https://learn.microsoft.com/en-us/azure/search/vector-search-ranking>
 - Cosmos DB vector search: <https://learn.microsoft.com/en-us/azure/cosmos-db/vector-database>
 - pgvector on Azure Database for PostgreSQL: <https://learn.microsoft.com/en-us/azure/postgresql/flexible-server/how-to-use-pgvector>
 - HNSW paper: <https://arxiv.org/abs/1603.09320>
 - GraphRAG (Microsoft Research): <https://microsoft.github.io/graphrag/>
 - RAGAS framework: <https://docs.ragas.io/>
 - Pinecone — Faiss vs alternatives explainer (vendor-neutral concept reference): <https://www.pinecone.io/learn/series/faiss/>
-

PHASE 4 — Agentic AI

Day 9 — Agent Fundamentals & Tool Use

Concepts

- What an agent is — an LLM that observes, plans, acts (via tools), and iterates until a goal is met.
- Agent patterns — **ReAct** (Reason + Act), **Plan-and-Execute**, **Reflexion**, **Tree-of-Thoughts**.
- Tool use / function calling — schema design, parallel calls, error handling.
- **Model Context Protocol (MCP)** — open standard for tool/data integration with LLMs.
- Memory — short-term (conversation), long-term (vector store of summaries/facts), episodic vs semantic.
- Planning, decomposition, and self-reflection.

Explanation

An **agent** is fundamentally an LLM in a loop: it sees the state (conversation + tool outputs), decides the next action (call a tool, ask a clarifying question, or finish), executes, observes the result, and continues until done or max-steps. The "intelligence" is in the loop + the prompt + the available tools.

ReAct interleaves Thought → Action → Observation → Thought → ... The thought is what makes it traceable and debuggable. Most modern agent frameworks are ReAct-based.

MCP (Model Context Protocol, originated by Anthropic, now adopted broadly including by Microsoft) is the "USB-C for LLM tools" — an open spec for how LLMs connect to data sources and tools via standardized servers. Instead of writing custom integrations per tool per framework, you write/use MCP servers that expose tools, resources, and prompts in a standard way. This is now a core part of the Microsoft Agent Framework.

Memory:

- *Short-term* — the conversation history in context (limited by window).
- *Long-term* — externalized: vector store of past conversations, structured facts (user preferences), or scratchpads. Retrieved into context as needed.
- *Episodic* (specific events) vs *semantic* (generalized knowledge) — useful distinction when building personalized assistants.

Hands-on

- Build a simple ReAct agent using OpenAI/Anthropic function calling. Give it 3 tools: web search, calculator, current date. Trace each step.
- Set up an MCP server (e.g., filesystem MCP) and connect a client. Observe how tools, resources, and prompts are exposed.

Interview Q&A

Q1. What's the difference between an LLM workflow and an agent? A. A **workflow** is a fixed sequence of LLM calls and code (e.g., parse → classify → route → respond) — deterministic structure, LLM does subtasks. An **agent** dynamically chooses its own steps and tools based on the situation; you give it a goal and tools, not a script. Trade-off: workflows are predictable, cheap, easy to debug, easy to certify; agents are flexible, expensive, harder to bound. Anthropic's "Building Effective Agents" guidance is correct here — most production needs are best served by workflows; reach for true agents only when the task space is too open to enumerate.

Q2. Explain ReAct. A. ReAct = Reasoning + Acting. The model is prompted to alternate between thinking out loud about what to do and taking an action (calling a tool). Each turn looks like:
Thought: "I need to find the customer's order status, so I should query the orders DB." *Action*:

`get_order(id=123)`). *Observation:* `{status: 'shipped', date: '...'}`. *Thought:* "Now I can answer the customer." *Final answer:* "...". The pattern makes the reasoning inspectable, supports error recovery, and is the basis of nearly every modern agent framework — implemented natively via tool calling in modern LLM APIs.

Q3. What is MCP and why does it matter? A. **Model Context Protocol** — an open standard (originated by Anthropic, now broadly adopted including Microsoft) that defines how LLM applications connect to external tools and data. An MCP server exposes **tools** (callable functions), **resources** (readable data), and **prompts** (reusable templates) via a standard interface (stdio or HTTP). Why it matters: instead of writing custom integrations per LLM client per tool — N×M problem — you write each integration once as an MCP server, and any MCP-capable client (Claude Desktop, Microsoft Agent Framework, etc.) can use it. It's becoming the standard plumbing for agentic systems.

Q4. How do you give an agent memory? A. Three layers. **Short-term:** the conversation thread in context. **Episodic long-term:** store summaries of past sessions in a vector index keyed by user/session; retrieve relevant ones at the start of a new conversation. **Semantic long-term:** structured facts (e.g., "User prefers metric units, lives in Berlin") in a KV or graph store; loaded into the system prompt deterministically. Watch for: token budget (don't dump entire history into context — summarize); staleness (facts get outdated); privacy (memory of user data needs retention controls and a way for users to delete it).

Official References

- Anthropic — "Building Effective Agents" (must-read): <https://www.anthropic.com/research/building-effective-agents>
 - OpenAI function calling guide: <https://platform.openai.com/docs/guides/function-calling>
 - Anthropic tool use guide: <https://docs.claude.com/en/docs/agents-and-tools/tool-use/overview>
 - Model Context Protocol spec: <https://modelcontextprotocol.io/>
 - MCP introduction: <https://modelcontextprotocol.io/introduction>
 - ReAct paper: <https://arxiv.org/abs/2210.03629>
 - Reflexion paper: <https://arxiv.org/abs/2303.11366>
 - LangGraph (stateful agents): <https://langchain-ai.github.io/langgraph/>
-

Day 10 — Multi-Agent Systems & Orchestration

Concepts

- Why multi-agent — specialization, separation of concerns, parallelism.

- Patterns — **sequential, concurrent, handoff, group chat, supervisor / orchestrator, hierarchical, Magentic-One.**
- Orchestration frameworks — Microsoft Agent Framework (the current Microsoft standard), LangGraph, OpenAI Swarm (deprecated → moved into Agents SDK), CrewAI, AutoGen (now folded into Agent Framework).
- State management, checkpointing, durability.
- Human-in-the-loop, approval gates.
- Inter-agent communication — A2A (Agent-to-Agent) protocol.

Explanation

A single agent struggles when tasks span very different skills (research + writing + code + review) or require parallelism. **Multi-agent** systems decompose:

- **Sequential** — pipeline of specialists (researcher → writer → editor).
- **Concurrent** — parallel agents work on independent subtasks; results merged.
- **Handoff** — agent transfers control when it hits its specialty boundary (e.g., support triage → billing specialist).
- **Group chat** — multiple agents converse; a manager or rule decides who speaks next.
- **Supervisor / orchestrator** — top-level agent decomposes, dispatches subtasks, integrates results.
- **Magentic-One** (Microsoft Research) — orchestrator + ledger pattern for general task solving with web/file/code agents.

State is the hardest part of production agents. Without durable state, a crash mid-conversation loses everything. Agent Framework provides checkpointing; LangGraph uses persistence backends (SQLite, Postgres, Redis).

Human-in-the-loop: for high-stakes actions (sending email, making payment, modifying production data) the agent should pause and request approval. Bake this into the workflow at design time, not as an afterthought.

Hands-on

- Build a 3-agent system: a researcher (web search), a writer (draft article), an editor (review and revise). Use Microsoft Agent Framework or LangGraph.
- Add a human approval gate before the final output is "published."

Interview Q&A

Q1. When should you go multi-agent instead of one big agent with many tools? A. Reach for multi-agent when: (a) the task has distinct subdomains where context and prompt would conflict if combined; (b) you want parallelism — multiple subtasks run concurrently; (c) different

agents need different models, tools, or guardrails (e.g., a "writer" with a creative model, a "reviewer" with a strict factual model); (d) the team's mental model of the system maps cleanly onto specialists. Don't go multi-agent for: (a) simple chains a workflow handles fine; (b) cost-sensitive deployments — each agent costs tokens; (c) latency-sensitive paths — coordination adds overhead. Start single-agent; split when evidence demands it.

Q2. Compare LangGraph, Microsoft Agent Framework, and CrewAI. A. **LangGraph** — stateful, graph-based; explicit nodes and edges; deep control over state, branching, persistence; strong for complex flows; Python-first; LangChain ecosystem. **Microsoft Agent Framework** (GA April 2026, successor to Semantic Kernel + AutoGen) — open source, Python + .NET, enterprise-grade with middleware, telemetry, durability, A2A and MCP support, deep Azure/Foundry integration, plus graph-based workflows. **CrewAI** — role-based abstraction (agents have roles, goals, backstories); fast to prototype role-playing crews; opinionated. My default for Microsoft-stack teams today is Microsoft Agent Framework; LangGraph if you're already deep in the LangChain ecosystem; CrewAI for rapid prototyping or non-technical-friendly setups.

Q3. How do you debug a misbehaving multi-agent system? A. (1) **Tracing first** — every prompt, tool call, and response logged with timestamps and a trace ID linking the chain. LangSmith, Foundry tracing, OpenTelemetry-based agent tracing all do this. (2) **Reproduce deterministically** — store the exact inputs and seeds; replay. (3) **Isolate** — run each agent in isolation with the captured inputs to find which one misbehaves. (4) **Inspect intermediate state** — agents pass messages; the bug is often in how state is constructed for the next agent, not in the model output itself. (5) **Add assertions** between hops — schema/format checks catch silent drift. (6) **Limit autonomy** while debugging — cap max steps; force human approval at boundaries.

Q4. What's the A2A protocol? A. **Agent-to-Agent** protocol — a standard for how agents discover and communicate with each other across systems, frameworks, even organizations. Where MCP standardizes agent-to-tool, A2A standardizes agent-to-agent. Microsoft Agent Framework supports A2A natively, enabling multi-vendor agent ecosystems. The motivation is similar to MCP: avoid $N \times M$ custom integrations. Still evolving; expect rapid change through 2026.

Official References

- Microsoft Agent Framework overview: <https://learn.microsoft.com/en-us/agent-framework/overview/>
- Microsoft Agent Framework — multi-agent orchestration: <https://learn.microsoft.com/en-us/agent-framework/concepts/multi-agent-orchestration>
- Microsoft Agent Framework GitHub: <https://github.com/microsoft/agent-framework>
- LangGraph documentation: <https://langchain-ai.github.io/langgraph/>
- LangGraph multi-agent tutorial: https://langchain-ai.github.io/langgraph/concepts/multi_agent/
- Magentic-One (Microsoft Research): <https://www.microsoft.com/en-us/research/articles/magentic-one-a-generalist-multi-agent-system-for-solving-complex->

tasks/

- CrewAI docs: <https://docs.crewai.com/>
 - OpenAI Agents SDK: <https://platform.openai.com/docs/guides/agents>
-

Day 11 — Production Agent Concerns

Concepts

- Cost and latency optimization — model routing (small for easy, big for hard), prompt caching, response streaming, batch where possible.
- Safety and guardrails — input/output filters, tool-use authorization, action allowlists, sandboxing code execution.
- Observability — tracing every LLM/tool call with OpenTelemetry, evaluation in CI/CD, golden datasets.
- Failure handling — retries, fallbacks, max-step limits, circuit breakers, timeouts.
- Determinism trade-offs — when to allow agent autonomy vs hard-code the path.
- Cost control — token caps per session, model tiering, caching identical sub-prompts.

Explanation

The gap between an agent demo and an agent in production is enormous and is almost entirely about the non-LLM concerns: **observability, cost, safety, latency, reliability**.

Model tiering: route simple sub-tasks (classification, extraction, routing decisions) to small/cheap models (GPT-4o-mini, Phi-3, Haiku); route hard reasoning to top-tier models. A well-designed agent can cost 5-10× less than one that uses the flagship model everywhere.

Prompt caching (supported by Anthropic, OpenAI, others) — repeated long prompt prefixes (system instructions, large context) are cached server-side and billed at a fraction. Critical for agents that reuse system prompts across thousands of calls.

Sandboxing code execution: if your agent runs Python or shell commands, sandbox them (Docker, gVisor, Firecracker, Azure Container Apps Dynamic Sessions). Never execute LLM-generated code on a host with access to your real environment.

Circuit breakers and max-step limits: an agent in a loop can burn \$1000 in tokens before anyone notices. Hard cap steps, set cost ceilings per session, alert on outliers.

Hands-on

- Add OpenTelemetry tracing to your agent (Foundry has built-in tracing; or use LangSmith/Phoenix).

- Implement a small/big model router — your agent classifies the difficulty of the user query first, then routes.
- Run your agent on 20 adversarial prompts (jailbreaks, prompt injections, off-topic) and document failures.

Interview Q&A

Q1. How do you keep agent costs under control? A. (1) **Model tiering** — cheap model for routing/extraction/easy steps; expensive model only when needed. (2) **Prompt caching** — reuse long system prompts via the API's cache feature; can reduce input costs by 50–90% on agentic flows. (3) **Trim context aggressively** — summarize old turns, drop tool outputs after they're consumed. (4) **Cap max steps per session** with a hard ceiling and alert. (5) **Token budgets** with per-user and per-session caps. (6) **Cache identical sub-prompts** at the application layer for deterministic sub-tasks. (7) **Batch where you can** — multiple independent sub-queries in parallel rather than sequential. (8) **Continuous cost monitoring** — dashboard with per-feature cost-per-request, alert on regressions.

Q2. How do you observe what an agent is doing? A. End-to-end **distributed tracing** with OpenTelemetry. Every LLM call, tool call, and agent transition emits a span with: prompt, response, model, latency, tokens, cost, parent span ID, custom attributes (user, session, feature). Use a tracing UI — Foundry tracing, LangSmith, Arize Phoenix, Datadog LLM Observability. On top of traces: structured logs (JSON), metrics (success rate, p95 latency, cost per session, tool error rate), and an eval pipeline that replays production traffic against new agent versions before deploy.

Q3. How do you handle a tool call that fails? A. Layered retry strategy: (1) **Deterministic retry** for transient failures (network, 5xx) with exponential backoff, capped attempts. (2) **Hand back to the agent** — the model sees the error in the observation and can reason about it (try a different tool, ask the user, give up). (3) **Fallback tool** — if `search_database` fails, try `cache_lookup`. (4) **Circuit breaker** — after N consecutive failures, mark the tool unavailable for X seconds. (5) **User-visible fallback** — if all fails, the agent responds with what it has plus a clear caveat. Never silently fail; never let the agent hallucinate a result that didn't come from the tool.

Q4. How do you let an agent take real-world actions safely (send email, make payment, etc.)? A. (1) **Principle of least privilege** — agent has narrow, scoped credentials per tool; can't access anything outside its scope. (2) **Allowlists** for sensitive parameters (only certain emails, certain amounts, certain SKUs). (3) **Human-in-the-loop approval** for any irreversible or high-stakes action — pause and present the proposed action to a user. (4) **Idempotency keys** for actions that mustn't repeat on retry. (5) **Audit log** of every action with prompt, decision, executor, outcome. (6) **Rate limits** per user, per session, per tool. (7) **Reversibility** where possible — prefer drafts over sends, holds over charges, soft-deletes over hard. (8) **Red-team** before launch with prompt injection and adversarial inputs against each action.

Official References

- Anthropic — prompt caching: <https://docs.claude.com/en/docs/build-with-claude/prompt-caching>
 - OpenAI — prompt caching: <https://platform.openai.com/docs/guides/prompt-caching>
 - Microsoft Foundry — agent observability: <https://learn.microsoft.com/en-us/azure/ai-foundry/concepts/trace>
 - OpenTelemetry GenAI semantic conventions: <https://opentelemetry.io/docs/specs/semconv/gen-ai/>
 - Azure Container Apps Dynamic Sessions (code interpreter sandbox): <https://learn.microsoft.com/en-us/azure/container-apps/sessions>
 - OWASP LLM Top 10 (2025): <https://genai.owasp.org/llm-top-10/>
 - Azure AI Content Safety: <https://learn.microsoft.com/en-us/azure/ai-services/content-safety/>
-

PHASE 5 — Microsoft Foundry & Microsoft Agent Framework

Day 12 — Microsoft Foundry (formerly Azure AI Foundry) Deep Dive

Concepts

- Foundry workspace model — projects, hubs, resources.
- **Model catalog** — Azure OpenAI, Llama, Mistral, Phi, Cohere, Hugging Face models; serverless vs managed deployment.
- **Foundry Agent Service** — hosted agents with built-in tools (file search, code interpreter, function tools, connected actions).
- **Prompt Flow** — visual + code orchestration; flows, variants, evaluations.
- **Foundry evaluations** — built-in metrics (groundedness, relevance, coherence, fluency, similarity); risk & safety metrics (harmful content, jailbreak, indirect attack); custom evaluators.
- **Grounding with your data** — connect to Azure AI Search, Cosmos DB, Blob; "On Your Data" pattern.
- **Content Safety** integration — hate, violence, sexual, self-harm, jailbreak, protected material, indirect attack.
- Deployment — managed online endpoints, serverless API endpoints.
- Networking — private endpoints, managed VNet, BYO VNet, customer-managed keys.

Explanation

Microsoft Foundry is the unified platform for building, deploying, and managing AI applications

on Azure. It consolidates what used to live across Azure ML, Azure OpenAI Studio, and Cognitive Services into one experience oriented around generative AI workflows.

Foundry Agent Service is the managed runtime for hosted agents — you bring your tools and prompts, Microsoft runs the orchestration with built-in state, tracing, and tool registry. It supports MCP, custom tools, and connected actions to other Azure resources.

Prompt Flow is the orchestration tool — visual DAG of nodes (LLM calls, Python functions, prompts, embeddings, vector lookups). Versionable, evaluable, deployable as an endpoint. Useful when you want a hybrid visual/code experience that BI-leaning teammates can read.

Evaluation is first-class — run any flow over a dataset, get a report on quality + safety + custom metrics, gate deployments. This is the single biggest production-readiness feature.

Hands-on

- Provision a Foundry project. Deploy GPT-4o and an embedding model.
- Build a RAG over your data using **Add Your Data** (Azure AI Search backing store).
- Build the same RAG as a **Prompt Flow** with explicit nodes.
- Run an **evaluation** on a 30-question dataset measuring groundedness, relevance, and similarity.
- Deploy the flow as a managed online endpoint and call it from a Python client.

Interview Q&A

Q1. How is Microsoft Foundry different from Azure ML? A. Azure ML is general-purpose ML — training pipelines, classical models, deployment of any model type. Microsoft Foundry is GenAI-first — model catalog with one-click frontier model access, agents, prompt flow, GenAI evaluation, content safety, grounding connectors. They share underlying compute, registries, and identity, but Foundry is the productive surface for LLM apps and agents while AML remains the right home for custom ML training. In practice on a modern team I see both used side-by-side.

Q2. Walk through how you'd build an enterprise RAG app on Foundry. A. Resources: a Foundry project (hub-scoped), Azure OpenAI deployments (chat + embeddings), Azure AI Search (vector + BM25 + semantic ranker), Document Intelligence for PDF parsing, ADLS Gen2 for raw documents. Pipeline: ADF or Foundry ingestion flow → parse with Document Intelligence → chunk → embed → write to AI Search with security-trimming metadata. App: Prompt Flow with retrieve → rerank → generate nodes, content safety on input and output, citation rendering. Evaluation: Foundry eval suite with groundedness + safety on a labeled set, run in CI. Deploy: managed endpoint behind APIM, authenticated, private endpoints, managed identity. Monitor: App Insights traces, drift on retrieval recall, cost dashboards.

Q3. What are Foundry's built-in evaluators and when do you use which? A. Quality:

groundedness (does the answer use only the provided context?), relevance (does it address the question?), coherence (is it well-structured?), fluency (natural language?), similarity (vs. a reference answer using LLM-as-judge). **Risk & safety:** hateful content, sexual, violent, self-harm, jailbreak detection, indirect attack detection, protected material. **Use:** groundedness + relevance for RAG; similarity if you have a reference dataset; safety metrics always on for production. Combine with custom evaluators (Python or prompt-based) for business-specific criteria like "answer in the company's tone" or "no PII leaked."

Q4. How do you handle networking and identity in Foundry for an enterprise deployment? A.

(1) **Managed identities** for all resource-to-resource auth — no keys. (2) **Private endpoints** on Azure OpenAI, AI Search, Storage, Cosmos, so data-plane traffic never traverses the public internet. (3) **Managed VNet** or **BYO VNet** for the Foundry workspace; outbound rules restrict where compute can reach. (4) **RBAC** at workspace, project, and resource scope with least privilege. (5) **Customer-managed keys (CMK)** for encryption where regulatory needs demand. (6) **Diagnostic logs** to Log Analytics and SIEM. (7) **Conditional access** on the portal for human users. (8) For multi-tenant SaaS, separate projects per tenant for blast-radius isolation.

Official References

- Microsoft Foundry documentation: <https://learn.microsoft.com/en-us/azure/ai-foundry/>
 - Microsoft Foundry — what is it: <https://learn.microsoft.com/en-us/azure/ai-foundry/what-is-azure-ai-foundry>
 - Microsoft Foundry — model catalog: <https://learn.microsoft.com/en-us/azure/ai-foundry/how-to/model-catalog-overview>
 - Foundry Agent Service: <https://learn.microsoft.com/en-us/azure/ai-foundry/agents/>
 - Prompt Flow: <https://learn.microsoft.com/en-us/azure/ai-foundry/how-to/prompt-flow>
 - Foundry evaluations: <https://learn.microsoft.com/en-us/azure/ai-foundry/concepts/evaluation-approach-gen-ai>
 - Azure AI Content Safety: <https://learn.microsoft.com/en-us/azure/ai-services/content-safety/>
 - Develop Generative AI apps learning path: <https://learn.microsoft.com/en-us/training/paths/develop-generative-ai-apps/>
 - Microsoft Foundry training portal: <https://learn.microsoft.com/en-us/training/azure/ai-foundry>
 - Hands-on exercises (GitHub): <https://github.com/MicrosoftLearning/mslearn-ai-studio>
-

Day 13 — Microsoft Agent Framework 1.0

Concepts

- Agent Framework = successor to Semantic Kernel + AutoGen (GA April 2026).
- Core abstractions — **Agent, Tool, ChatAgent, Workflow, State, Memory, Middleware.**
- **Workflows** — graph-based orchestration; nodes can be agents, code, or other workflows.
- **Multi-agent orchestration patterns** — sequential, concurrent, handoff, group chat, **Magentic-One.**
- **Memory & context providers** — session state, durable checkpointing, human-in-the-loop pause/resume.
- **Tool integration** — function tools, MCP, OpenAPI specs, A2A.
- **Observability** — OpenTelemetry-based tracing out of the box; integrates with Foundry tracing and Application Insights.
- **Migration** — from Semantic Kernel and from AutoGen.

Explanation

The Microsoft Agent Framework is the single Microsoft-blessed SDK for building agents going forward. It unifies what was previously split across Semantic Kernel (enterprise SDK) and AutoGen (research-oriented multi-agent framework), keeping the best of both.

Two primary building blocks:

- **Agents** — single stateful execution units that take input, use tools, and produce output. Backed by any chat completion or Responses model. Include middleware hooks for logging, auth, rate limiting.
- **Workflows** — graph-based DAGs where nodes are agents (or code, or sub-workflows). Edges define how state flows. Provide checkpointing for long-running and human-in-the-loop scenarios.

This separation is intentional: a single agent is great for self-directed tasks; a workflow is great for explicitly orchestrated multi-step processes with predictable structure.

Languages: Python and .NET at near parity. Open source under MIT. Deep integration with Microsoft Foundry, Azure OpenAI, AI Search, but also supports OpenAI, Anthropic, Hugging Face, Ollama, and others.

Migration: If you have Semantic Kernel or AutoGen code, Microsoft provides migration guides and assistants. New projects should start with Agent Framework.

Hands-on

- Install Agent Framework (`pip install agent-framework`) or NuGet equivalent).

- Build a single ChatAgent with two custom tools (e.g., a calculator and a weather lookup). Wire to Azure OpenAI.
- Build a sequential workflow: researcher agent → writer agent → reviewer agent with a human-approval node before output.
- Add OpenTelemetry tracing and view in Foundry tracing UI.

Interview Q&A

Q1. Why was Microsoft Agent Framework created when Semantic Kernel and AutoGen existed? A. Developers wanted both the enterprise-grade features of Semantic Kernel (telemetry, middleware, state, security, broad model support) and the multi-agent innovation of AutoGen (orchestration patterns, autonomous collaboration) in one stack — the team running both projects merged them. Agent Framework adds explicit graph-based workflows on top, plus first-class MCP and A2A support, plus durability/checkpointing for long-running scenarios. It went GA in April 2026 with stable APIs and long-term support; Semantic Kernel 1.x continues to be maintained for existing users for at least a year after Agent Framework GA.

Q2. Compare Workflows vs Agents in the framework — when do you use which? A. **Agent** is the right choice when a single LLM with tools and memory can solve the task autonomously — e.g., a customer support assistant. **Workflow** is the right choice when the task has a known structure with multiple steps you want to control explicitly — e.g., "ingest a contract → extract clauses → check against playbook → flag risks → summarize." Workflows give you deterministic structure, parallel branches, conditional edges, persistence at each node, and easier debugging. Many production systems are a workflow whose nodes are agents — best of both.

Q3. How does Agent Framework integrate with MCP? A. Natively — MCP servers can be plugged in as tool sources. An agent can discover available tools from an MCP server, call them like native tools, and consume resources. This means you can reuse the growing MCP ecosystem (file system, GitHub, Slack, databases, custom corp tools) without writing custom adapters. Combined with A2A for inter-agent communication, the framework participates in a cross-vendor agent ecosystem rather than being a closed environment.

Q4. How do you handle durability and human-in-the-loop in a multi-step Agent Framework workflow? A. The framework provides **checkpointing**: workflow state is persisted at each node, so on crash or restart the workflow resumes where it left off. For **human-in-the-loop**, you add a node that pauses execution and emits an event; an external system (UI, email, Teams) prompts the human; on response the workflow resumes with the human input fed in as state. Storage backends are pluggable (in-memory for dev, durable for prod — typically a database). This pattern is essential for long-running processes (loan approval that takes 3 days, agent that waits for an SLA window).

Official References

- Agent Framework documentation: <https://learn.microsoft.com/en-us/agent-framework/>

- Agent Framework overview: <https://learn.microsoft.com/en-us/agent-framework/overview/>
 - Agent Framework GitHub: <https://github.com/microsoft/agent-framework>
 - Develop AI Agents on Azure learning path: <https://learn.microsoft.com/en-us/training/paths/develop-ai-agents-azure/>
 - Semantic Kernel → Agent Framework migration: <https://learn.microsoft.com/en-us/agent-framework/migration-guide/>
 - Semantic Kernel docs (legacy reference): <https://learn.microsoft.com/en-us/semantic-kernel/>
 - AI Agents for Beginners (course): <https://microsoft.github.io/ai-agents-for-beginners/>
 - Magentic-One paper / write-up: <https://www.microsoft.com/en-us/research/articles/magentic-one-a-generalist-multi-agent-system-for-solving-complex-tasks/>
-

PHASE 6 — LLMOps & Responsible AI

Day 14 — LLMOps, Evaluation, Monitoring, Governance

Concepts

- LLMOps vs MLOps — what's the same, what's new.
- The LLMOps lifecycle — prompt/model versioning, eval-driven development, staged rollout (shadow → canary → full), continuous monitoring, feedback loops.
- Evaluation in CI/CD — golden datasets, automated evals, regression gates, A/B testing.
- Monitoring in production — quality (groundedness, drift), safety (content safety violations), operational (latency, error rate), cost (per-feature, per-tenant).
- Responsible AI — fairness, transparency, accountability, security, privacy, reliability, inclusiveness.
- Microsoft's Responsible AI Standard + Foundry's responsible AI features.
- Governance — model approval workflows, audit trails, RAI documentation.

Explanation

LLMOps inherits MLOps practices (versioning, CI/CD, monitoring, governance) but adds layers specific to LLMs:

- **Prompts are code** — version-controlled, tested, evaluated, gated on regression.
- **Evals are non-negotiable** — without them, you can't ship safely.
- **Stochastic outputs** require statistical evaluation, not unit tests.

- **Drift detection** is more nuanced — input drift (user behavior), output drift (model upgrades on the provider side), retrieval drift (corpus changes).
- **Cost as a first-class metric** — model usage scales with traffic in ways traditional ML doesn't.
- **Safety monitoring** — every production response can be checked by a content-safety classifier; violations alerted.

Eval-driven development is the practice that distinguishes mature LLM teams: every change (prompt, model, retriever) is measured against a versioned eval set; releases are gated on regressions; production traffic is sampled back into the eval set.

Responsible AI in Azure is operationalized via Foundry's risk & safety evaluators, Content Safety, model cards, transparency notes, and audit logs. Microsoft publishes a Responsible AI Standard that teams use as a checklist for new AI workloads.

Hands-on

- Build a CI evaluation: a Python script that runs your RAG/agent against a 50-question dataset, computes groundedness + relevance + safety, and fails the build on regression vs a baseline.
- Wire up production monitoring: log every prompt, response, latency, token count, content-safety flags to App Insights. Build a Grafana/Power BI dashboard.
- Run a shadow deployment of a new model version on 10% of real traffic, compare to the current production model on the eval set.

Interview Q&A

Q1. How is LLMOps different from MLOps? A. Both share the CI/CD, versioning, monitoring spine. LLMOps adds: (1) **Prompts as artifacts** — first-class versioned, tested, evaluated. (2) **Stochastic evaluation** — outputs aren't deterministic; you need statistical comparison over a dataset, not unit asserts. (3) **External model dependency** — your "model" may be a vendor API that silently changes; pin model versions and run regression evals on model upgrades. (4) **Retrieval as a moving part** — corpus changes affect quality; treat the index as code. (5) **Cost as a primary metric** — tokens × calls scales with traffic, often non-linearly. (6) **Safety evals continuous** — content safety classifiers on production outputs, not just at eval time.

Q2. What does a golden evaluation set look like, and how do you build one? A. A versioned set of (input, expected outcome) pairs — typically 50–500 — that covers (a) common cases, (b) edge cases, (c) known failure modes, (d) adversarial inputs, and (e) regression bug patterns. Build it incrementally: seed from product spec, expand from real production failures, periodically refresh. Score with a mix of deterministic (schema, citations present) and LLM-as-judge metrics calibrated to human ratings on a small subset. Every change is gated against the set; the set itself

is reviewed quarterly. Anti-patterns: a static set that doesn't grow; "easy" sets that everything passes; or sets so large that running them is slow.

Q3. How do you monitor a RAG/agent in production? A. Four layers: (1) **Operational** — p50/p95/p99 latency, throughput, error rate, retry rate per tool. (2) **Quality** — sampled live evals (groundedness, relevance), user thumbs/feedback rate, follow-up question rate (proxy for failure). (3) **Safety** — content safety flag rate, jailbreak detection rate, blocked content count. (4) **Cost & usage** — tokens per request, cost per session, per-tenant cost, anomaly alerts. Dashboards in App Insights / Grafana / Power BI; alerts via Azure Monitor; weekly review of failure samples by humans. The key: production isn't fire-and-forget — schedule the review.

Q4. What's in Microsoft's Responsible AI Standard and how do you apply it? A. Six principles: **fairness, reliability & safety, privacy & security, inclusiveness, transparency, accountability**. Applied via a structured process: (1) **Impact assessment** before development — identify stakeholders, intended use, foreseeable misuse, sensitive uses. (2) **Data documentation** — data origin, quality, representativeness. (3) **Model documentation / model card** — capabilities, limitations, evaluation results, intended deployment. (4) **Evaluation** including safety + fairness across demographic slices. (5) **Mitigations** — content safety, RLHF, human review for high-risk use. (6) **Transparency notes** for users — what the system does and doesn't do. (7) **Monitoring + incident response** post-deployment. Foundry surfaces many of these as tooling; the standard itself is the policy framework.

Official References

- Azure ML — MLOps guide: <https://learn.microsoft.com/en-us/azure/machine-learning/concept-model-management-and-deployment>
 - Microsoft Foundry — evaluation overview: <https://learn.microsoft.com/en-us/azure/ai-foundry/concepts/evaluation-approach-gen-ai>
 - Microsoft Foundry — monitor your application: <https://learn.microsoft.com/en-us/azure/ai-foundry/how-to/monitor-applications>
 - Microsoft Responsible AI: <https://www.microsoft.com/en-us/ai/responsible-ai>
 - Microsoft Responsible AI Standard (v2 PDF on Microsoft's site): <https://www.microsoft.com/en-us/ai/principles-and-approach>
 - Azure AI Content Safety overview: <https://learn.microsoft.com/en-us/azure/ai-services/content-safety/overview>
 - MLflow LLMOps: <https://mlflow.org/docs/latest/llms/index.html>
 - OWASP LLM Top 10: <https://genai.owasp.org/llm-top-10/>
 - NIST AI Risk Management Framework: <https://www.nist.gov/itl/ai-risk-management-framework>
-

PHASE 7 — Capstone & Interview Prep

Day 15 — Capstone Build + Mock Interviews

Goal

Tie everything together by building one cohesive system and rehearsing the interview narrative around it.

Capstone project (pick one)

Option A — Enterprise RAG Assistant A document-grounded assistant over a corpus you choose (your team wiki, your industry's regulations, your reading library). Must include: ingestion pipeline, Azure AI Search hybrid + semantic ranker, Foundry Prompt Flow with safety, citations, evaluation suite, deployed endpoint, monitoring dashboard.

Option B — Multi-Agent Research Assistant A research workflow using Microsoft Agent Framework with 3+ agents (researcher with web/search tool, summarizer, fact-checker), human-in-the-loop approval before final output, OpenTelemetry tracing, evaluation harness.

Option C — Agentic Data Insights An agent that connects to your data warehouse (Fabric Warehouse, Synapse, or Azure SQL), takes natural language questions, generates SQL, executes safely (read-only role), and presents results with Power BI / chart visuals. Includes guardrails on SQL generation, query cost limits, and explanation of results.

What to produce

- Working code in GitHub.
- Architecture diagram (you've done many — apply the same rigor).
- README with design decisions and trade-offs.
- Evaluation results — a table showing baseline vs your version on quality metrics.
- A 5-minute walkthrough video or notes.

Interview narrative practice

Prepare a 90-second "tell me about your AI project" pitch covering: **problem** → **architecture** → **key trade-offs** → **measurable outcome** → **what you'd do differently**. Rehearse it.

Common interview formats to expect (for senior AI engineer / AI architect roles):

1. **System design** — "Design a RAG system for an insurance company's claims docs." Use what you built on Day 7-8 + 12.
2. **Agentic design** — "Design a multi-agent customer support system." Use Days 9-11 + 13.
3. **Trade-off questions** — "RAG vs fine-tuning?" "When wouldn't you use an agent?" Use Days 5, 6, 9.

4. **Production / scale** — "How do you keep this under \$X/month?" "How do you monitor for hallucination?" Use Days 11, 14.
5. **Responsible AI** — "How do you ensure the system is fair, safe, and compliant?" Use Day 14.

Mock questions to rehearse out loud

1. *Walk me through how you'd design a customer-facing AI assistant for a regulated industry.*
2. *Your RAG system is hallucinating 15% of the time. What's your debugging process?*
3. *Your agent costs \$0.50 per session and management says it must be \$0.10. Walk me through optimization options.*
4. *We have 10 million documents updated daily. Design the ingestion + indexing pipeline.*
5. *Explain the difference between a workflow and an agent, and tell me about a time you chose one over the other.*
6. *How would you migrate a Semantic Kernel application to Microsoft Agent Framework?*
7. *A user reports the agent did something inappropriate. How do you investigate, what do you change, and how do you prevent regressions?*
8. *We want to use a fine-tuned model for tone, a RAG for facts, and an agent for orchestration. Sketch the architecture.*

Final references — interview prep

- Microsoft AI Engineer career path: <https://learn.microsoft.com/en-us/training/career-paths/ai-engineer>
- Microsoft Certified: Azure AI Engineer Associate (AI-102) — being replaced by AI-103 in 2026: <https://learn.microsoft.com/en-us/credentials/certifications/azure-ai-engineer/>
- Microsoft AI Show on YouTube: <https://www.youtube.com/@MicrosoftDeveloper>
- Anthropic "Building Effective Agents": <https://www.anthropic.com/research/building-effective-agents>
- OpenAI "A Practical Guide to Building Agents": <https://cdn.openai.com/business-guides-and-resources/a-practical-guide-to-building-agents.pdf>

Appendix A — Bookmark Bar (the 20 links worth keeping)

#	Resource	URL
---	----------	-----

1	Microsoft Foundry docs	https://learn.microsoft.com/en-us/azure/ai-foundry/
---	------------------------	---

2	Microsoft Agent Framework docs	https://learn.microsoft.com/en-us/agent-framework/
---	--------------------------------	---

#	Resource	URL
3	Microsoft Agent Framework GitHub	https://github.com/microsoft/agent-framework
4	Azure OpenAI docs	https://learn.microsoft.com/en-us/azure/ai-services/openai/
5	Azure AI Search docs	https://learn.microsoft.com/en-us/azure/search/
6	Azure AI Content Safety	https://learn.microsoft.com/en-us/azure/ai-services/content-safety/
7	MS Learn — Develop AI Agents on Azure	https://learn.microsoft.com/en-us/training/paths/develop-ai-agents-azure/
8	MS Learn — Develop Generative AI Apps	https://learn.microsoft.com/en-us/training/paths/develop-generative-ai-apps/
9	Foundry hands-on exercises	https://github.com/MicrosoftLearning/mslearn-ai-studio
10	OpenAI API docs	https://platform.openai.com/docs
11	Anthropic API docs	https://docs.claude.com/en/api
12	Anthropic "Building Effective Agents"	https://www.anthropic.com/research/building-effective-agents
13	Model Context Protocol	https://modelcontextprotocol.io/
14	LangChain Python	https://python.langchain.com/
15	LangGraph	https://langchain-ai.github.io/langgraph/
16	LlamaIndex	https://docs.llamaindex.ai/
17	Hugging Face Transformers	https://huggingface.co/docs/transformers/
18	RAGAS	https://docs.ragas.io/
19	MTEB leaderboard	https://huggingface.co/spaces/mteb/leaderboard
20	OWASP LLM Top 10	https://genai.owasp.org/llm-top-10/

Appendix B — Seminal Papers (read at least the abstract + conclusion of each)

Concept	Paper	Link
Transformer	Attention is All You Need (Vaswani et al., 2017)	https://arxiv.org/abs/1706.03762
BERT	Pre-training of Deep Bidirectional Transformers (Devlin et al., 2018)	https://arxiv.org/abs/1810.04805
GPT-3 / scaling	Language Models are Few-Shot Learners (Brown et al., 2020)	https://arxiv.org/abs/2005.14165
RLHF	Training language models to follow instructions with human feedback (Ouyang et al., 2022)	https://arxiv.org/abs/2203.02155
Chain-of-Thought	(Wei et al., 2022)	https://arxiv.org/abs/2201.11903
ReAct	Synergizing Reasoning and Acting in Language Models (Yao et al., 2022)	https://arxiv.org/abs/2210.03629
RAG	Retrieval-Augmented Generation (Lewis et al., 2020)	https://arxiv.org/abs/2005.11401
LoRA	Low-Rank Adaptation of Large Language Models (Hu et al., 2021)	https://arxiv.org/abs/2106.09685
QLoRA	Efficient Finetuning of Quantized LLMs (Dettmers et al., 2023)	https://arxiv.org/abs/2305.14314
DPO	Direct Preference Optimization (Rafailov et al., 2023)	https://arxiv.org/abs/2305.18290
Lost in the Middle	(Liu et al., 2023)	https://arxiv.org/abs/2307.03172
HyDE	Precise Zero-Shot Dense Retrieval (Gao et al., 2022)	https://arxiv.org/abs/2212.10496
Reflexion	Reflexion: Language Agents with Verbal Reinforcement Learning	https://arxiv.org/abs/2303.11366
Toolformer	Language Models Can Teach Themselves to Use Tools (Schick et al., 2023)	https://arxiv.org/abs/2302.04761

Closing Notes

On pace: This plan is intentionally aggressive. If a topic clicks faster (likely Days 1–3 given your background), use the time to go deeper into Days 9–13. If a topic is new (likely Days 9–13), don't skip the hands-on — reading alone doesn't build interview confidence.

On your Azure background: Lean into it in interviews. Many candidates can describe RAG abstractly. Few can describe how they'd actually deploy it on Azure with private endpoints, managed identities, ADLS Gen2 layout, and AI Search hybrid retrieval — and have actually built data pipelines that work at scale. That's your moat. Tell stories that pair your data engineering rigor with your new GenAI fluency.

On what to skip: You can skim Day 1 if you've done classical ML before. You can skip ahead on Day 3 if you've already studied transformers. But do not skip Days 9–11 or 13 — agentic AI is where the market is heading and where the highest-leverage interview signals live in 2026.

On staying current: AI moves fast. After Day 15, subscribe to: Microsoft Tech Community AI blog, Anthropic news, OpenAI news, Hugging Face Daily Papers, and one good newsletter (The Batch by Andrew Ng, or Latent Space). Keep a learning hour weekly — the field will change underneath you otherwise.

Good luck.